

Software Engineering

22/07/2026

INTRODUCTION

Program: The executable instructions written in programming languages that tells the hardware exactly what to do. A program is a set of instructions that can be executed by a computer to do a specific task.

Software: Software is a collection of programs, procedures, data structures and the associated documentation required to make a system in functions and evolve. It acts as both products and vehicle for delivering computing power across hardware networks.

Data structures: The structured formats and algorithms that allows the program to efficiently manipulate, store and process the information/data.

Documentation: The technical manuals requirements design blue prints and user guides that allows to manage and scale the system for long term.

Software Engineering: It is the systematic and disciplined application of engineering principles to design, develop, test and maintain of software.

Like any other engineering discipline, software engineering makes heavy use of the knowledge that has accrued from the experiences of a large number of practitioners. These past experiences have been systematically organised and wherever possible theoretical significance (basis to the empirical observations) could be provided. The past experiences have been modified but there is no theoretical support, then the experience is the thumb of rule.

An appropriate solution is chosen based on the trade-off

- Cost
- Time
- Feature/functionality

23/07/2026

EVOLUTION OF AN ART INTO AN ENGINEERING DISCIPLINE

The early programmers used an *ad hoc* programming style. This style of program development is now variously being referred to as *exploratory*, *build and fix*, and *code and fix* styles.

In a build and fix style, a program is quickly developed without making any specification, plan, or design. The different imperfections that are subsequently noticed are fixed.

The exploratory programming style is an informal style in the sense that there are no set rules or recommendations that a programmer has to adhere to—every programmer himself evolves his own software development techniques solely guided by his own intuition, experience and personal preferences.

Roughly 31% of the software development projects succeed meaning thereby, they are developed on time. Within the budget and with required functionalities.

69% are classified as failure due to over budgeting, missing deadlines or not providing all features as promised or cancelled.

The Standish Group (IT research and advisory firm based in Boston) published the CHAOS biennial study analysing why IT and software development projects succeed, fail or struggle in an organization worldwide, it says 19% outright failure and 50% challenged.

SOFTWARE DEVELOPMENT PROJECTS

Program versus Products

Many toy/small software are being developed by individuals such as students for their classroom assignments are meant for their personal use, these toy software does not have any supporting documents such as development document, maintenance manual, design document, test documents, etc., we call these toy software as *programs*.

Professional software has a large number of users, it is systematically designed, carefully implemented, and thoroughly tested. In addition, a professionally written software usually consists not only of the program code but also of all associated documents such as requirements specification document, design document, test document, users' manuals, maintenance related documents etc. They are called software products.

A professional software is developed by a group of software developers working together in a team. It is therefore necessary for them to use some systematic development methodology.

CAR Hoare [1994] observed that rigorously using software engineering principles to develop small/toy programs is very much like employing civil and architectural engineering principles to build sand castles for children to play

TYPES OF SOFTWARE DEVELOPMENT PROJECTS

A software development company is typically structured into a large number of teams that handle various types of software development projects. These software development projects concern the development of either software product or some software service.

- **Software products**

A variety of software such as Microsoft's Windows, Oracle DBMS etc., these software are available off-the-shelf for purchase and are used by a diverse range of customers. These are called *generic software products*. When a software development company wishes to develop a generic product, it first determines the features or functionalities that would be useful to a large cross section of users across different domains. Based on these, the development team draws up the product specification. Many companies find it advantageous to develop *product lines* that target slightly different market segments based on variations of essentially the same software. For example, Microsoft targets desktops and laptops through its *Windows 10/11*

operating system, while it targets high-end mobile handsets through its *Windows mobile* operating system, and targets servers through its *Windows server 22/21* operating system.

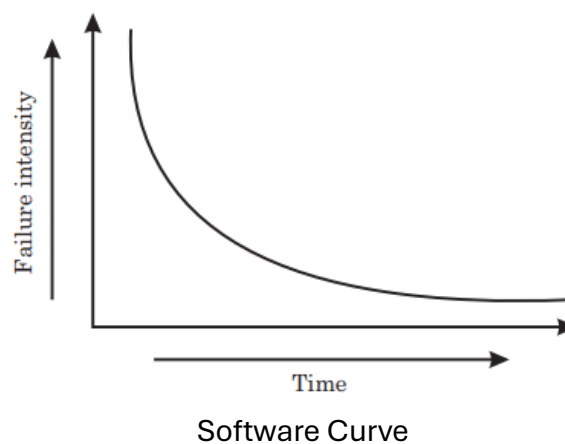
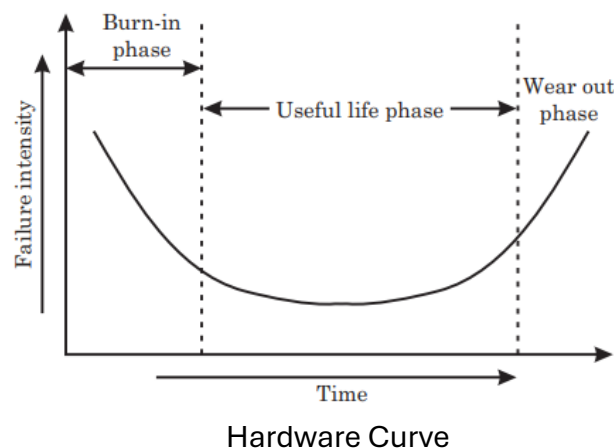
24/07/2026

- Software Services

A software service usually involves either development of a *customised software* or development of some specific part of a software. A *customised software* is developed according to the specification drawn up by one or at most a few customers. These need to be developed in a short time frame (typically a couple of months), and at the same time the development cost must be low.

For example – One university wants to automate its activities, registration, collection, grading, attendance etc. If the university wants to have automation for finance and purchase, then these functionalities can be developed by hiring some firms.

In a customised software development project, a large part of the software is reused from the code of related software that the company might have already developed. Usually, only a small part of the software that is specific to some (desired) client is developed and rest is taken from the existing code.

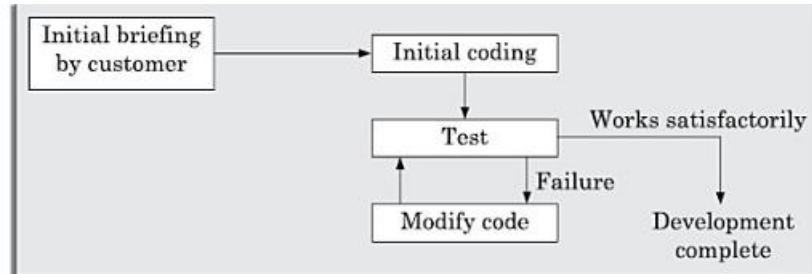


27/07/2026

EXPLORATORY STYLE OF SOFTWARE DEVELOPMENT

An exploratory development style can be successful when used for developing very small programs (1000-2000 LOC), and not for professional software (Worse).

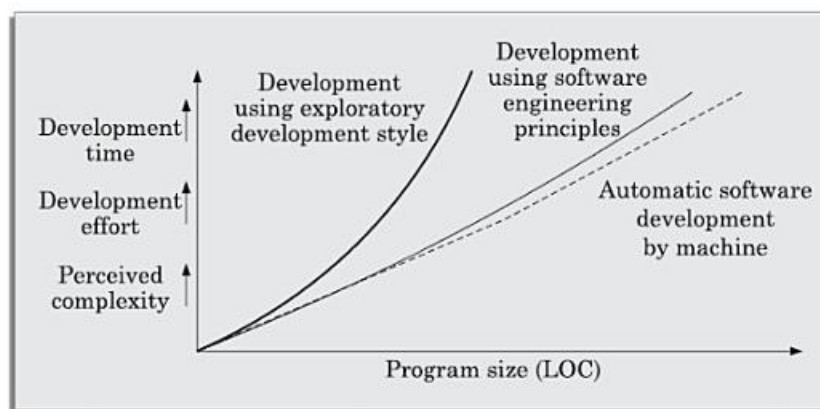
The exploratory style imposes no rules a typical development starts after an initial briefing from the customer. Based on this briefing, the developers start coding to develop a working program. The software is tested and the bugs found are fixed. This cycle of testing and bug fixing continues till the software works satisfactorily for the customer.



WHAT IS WRONG WITH THE EXPLORATORY STYLE OF SOFTWARE DEVELOPMENT?

Using the exploratory style, you may easily solve a problem that requires writing only 1000 or 2000 lines of source code. But, if you are asked to solve a problem that would require writing one million lines of source code, you may never be able to complete it using the exploratory style; irrespective of the amount time or effort you might invest to solve it.

For large problems, the exploratory style will take too much time and also code, impossible to comprehend the human memory.



If the problem needs 1 million of code say, then software engineering principle takes effects which is almost linear.

The exploratory style requires the effects which increases exponentially with the size of code. This is due to human cognitive limitations.

PERCEIVED PROBLEM COMPLEXITY: AN INTERPRETATION BASED ON

HUMAN COGNITION MECHANISM

The human memory can be thought to consist of two distinct parts: short-term and long-term memories.

Short-term memory: It is like a RAM in computer it can store 7 ± 2 items at a time, it can store the information few seconds, and at most for a few minutes. To remember it, we need to refresh it. The information stored in the short-term memory is immediately accessible for processing by the brain.

Long-term memory: Unlike the short-term memory, the size of the long-term memory is not known to have a definite upper bound. The size of the long-term memory can vary from several million items to several billion items, largely depending on how actively a person exercises his mental faculty.

An item once stored in the long-term memory, is usually retained for several years.

But, how do items get stored in the long-term memory? Items present in the short-term memory can get stored in the long-term memory either through large number of refreshments (repetitions) or by forming links with already existing items in the long-term memory.

When the number of details (or variables) that one has to track to solve a problem increases beyond seven, it exceeds the capacity of the short-term memory and it becomes exceedingly more difficult for a human mind to grasp the problem.

29/07/2026

PRINCIPLES DEPLOYED BY SOFTWARE ENGINEERING TO OVERCOME HUMAN COGNITIVE LIMITATIONS

Two important principles that are deployed by software engineering to overcome the problems arising due to human cognitive limitations are—abstraction and decomposition.

Abstraction: Abstraction refers to construction of a simpler version of a problem by ignoring the details. The principle of constructing an abstraction is popularly known as *modelling* (or *model construction*).

A single level of abstraction can be sufficient for rather simple problems. However, more complex problems would need to be modelled as a hierarchy of abstractions.

Decomposition: It is also known as the *divide and conquer* principle/approach. It breaks the problem into many small independent parts. The parts are taken up one by one and solved separately and finally they are integrated.

SOFTWARE APPLICATION DOMAINS

Today, seven broad categories of software present continuing challenges for the software engineers.

System Software: A collection of programs written to service other programs. Some system software are compilers, editors that process complex but determinate information structures.

A determinate software is one in which the order and timing of inputs, processing and outputs are predictable. If this cannot be predicted, then such system software is called indeterminate like Operating System, drivers, networking software, telecommunications processors etc.

Application Software: They are standalone programs that solve a specific business need. Applications software process our technical data in a way that facilitates business operations or management or technical decision making.

Engineering/Scientific Software: They are a broad array of number crunching programs that range from astronomy to volcanology, genetic analysis to meteorology.

Embedded Software: They reside within a product or system and are used to implement and control features and functions for end users and for the system itself. They can perform limited and esoteric functions like keypad control for microwave oven.

Product Line Software: They compose of reusable components which are designed to provide specific capabilities for use by many different customers.

Web/ Mobile Software: The category spans a wide array of applications. They encompass browser-based applications, cloud computing software that reside on mobile devices.

Artificial Intelligence Software: These software make use of heuristics to solve complex problems that are not amenable to regular computation or straight forward analysis

IEEE has developed the following definition for software engineering: *Software Engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software, i.e., the application of engineering to software.*

The foundation for software engineering is the process which forms the basis for management, control of software projects and establishes the context in which technical methods are applied, work products (model, documents, data, report etc.) are produced milestones are established, quality is ensured and changes are properly managed.

Software Methods: They provide the technical “how-to’s” for building software which include communications, requirement analysis, design modelling, program construction, testing and maintenance.

30/07/2026

Legacy Software: Dayani-Fard describe legacy software in the following way:

Legacy software systems were developed decades ago and have been continually modified to meet changes in business requirements and computing platforms. The proliferation of such systems is causing headaches for large organizations who find them costly to maintain and risky to evolve.

They are basically old computers programs that people and business uses, even though newer and better options exist.

Their key traits include being outdated, hard to fix and tied to old hardware.

WHY THE CORPORATE USE LEGACY SYSTEMS

1. **Cost:** Buying and selling up a brand-new system is very costly.
2. **Core Work:** The old programs still do its main job without failing.

3. **Risk:** Changing system can cause major data loss on work stages.

THESE SYSTEMS HAVE FOLLOWING RISKS:

1. **High Cost:** Finding bugs and finding expert is very difficult.
2. **Safety Risk:** Old system lacks modern security making them easy target for hackers.
3. **Slow Speed:** Workers waste time around the system instead of doing new tasks.

Software Process: A process is a collection of activities, actions and tasks that are performed when some work product is to be created.

An activity strives to achieve a broad objective for example, communication with stakeholders, which is applied irrespective of domain, size of project, complexity of efforts, or degree of rigour with which software engineering is to be applied.

An action (which is an architectural design) encompasses a set of tasks that produce a major product which is termed as Architectural Model.

A task focuses on a small but well-defined objectives, for examples, conducting a unit testing that produces a tangible outcome.

In context of software engineering, a process is not a rigid prescription but an adaptable approach that enables the software development team to pick and choose the appropriate set of work, action and tasks with the intent to deliver software on time with required functionality.

Process Framework: A process framework establishes the foundation for a complete software engineering process by identifying a small number of framework activities that are applicable to all software projects, regardless of their size or complexity.

In addition, the process framework encompasses a set of umbrella activities that are applicable across the entire software process. The umbrella activities in software engineering are supporting task that helps in the entire software development to manage progress, quality, change and risk. Examples are risk management. They are called umbrella because they cover and protect the whole project from start to finish.

31/07/2026

A generic process framework for software engineering encompasses 5 activities.

1. **Communication:** Before starting any technical work, it is critically important to communicate and collaborate with the customers and other stakeholders like business managers, end users, software engineers, support persons etc. The intent is to understand the stakeholders' objectives for the project and to gather requirements that help to define software features and functionalities.
2. **Planning:** Any complicated journey can be simplified if any map exists. A software project is a complicated journey and the planning activity creates a map that help to guide the team as it makes a journey. The map called a software project plan, defines the software work by describing the technical tasks to be constructed, the risks that are likely, the resources that will be required, the work product to be produced and the work schedule.

3. Modelling: We create a sketch of the things so that we can understand the big picture, what it will look like architecturally, how the constituent parts fits together and many other characteristics. If required we refine the sketch into greater detail in an effect to be better understand the problem and how we are going to solve it.

A software engineer does the same thing by creating models to better understand software requirement and the design that will achieve those requirements.

4. Construction: What we design must be built this activity combines code generation and testing that is required to uncover errors in the code.

5. Deployment: The software is delivered to the customer who evaluates it and provides feedback based on the evaluation.

These 5 activities can be used during development of the program, creation of web applications, complex systems. The details of the software process will quite different but the framework activities will remain the same.

The software engineering framework activities are complemented by a number of umbrellas activities. Generally, they are applied throughout the project development and help a software team manage and control progress, quality, change and risks.

Typically, the umbrella activities include

- a. Software Project Tracking and Control:** This allows software team to assess the progress against the project plan and take necessary action to maintain the schedule.
- b. Risk Management:** This assesses the risks that may affect the outcome of the project or quality of the product.
- c. Software Quality Assurance:** This defines and conducts the activities required to ensure software quality.
- d. Technical Review:** This assesses software product is an effort to uncover and remove the errors before they are propagated to the next activity.
- e. Measurement:** This defines and collects process, projects and product measures that assists the team in delivering the software that meets stakeholders' needs can be used in conjunction with all other framework and umbrella activities.
- f. Software Configuration Management:** This manages the effects of change throughout the software process.
- g. Reusability Management:** This defines the criteria for work product reuse and establishes mechanisms to achieve reusable components.

h. Work Product Preparation and Productions: This encompasses the activities required to create work products such as models, documents, logs, forms, lists etc.

03/08/2026

SOFTWARE ENGINEERING PRACTICES

In the classic book “How to solve it” written by George Polya, before modern computer existed, outlined the *essence of problem solving* and consequently the *essence of software engineering practices*.

- Understand the problem (Communication and analysis)
- Plan a solution (Modelling and software design)
- Carry out plan (Code generation)
- Examining the result for accuracy (Testing and quality assurance)

1. Understanding the problem: It is sometimes difficult to admit but most of us suffer from hubris, when we are presented with a problem. We listen for a few seconds and then think – “Oh yeah, I understand, let us go for solution” unfortunately it is not always easy. It is worth spending a little time answering a few simple questions.

- Who has the stake in the solution to the problem?
- What are the unknowns? What data, features and functions are required to properly solve the problem?
- Can the problem be compartmentalized? Is it possible to represent the problem into smaller problem which may be easily solvable?
- Can the problem be represented graphically? Can an analysis model be created?

2. Plan the solution: Now you have understood the problem or you think so and cannot wait to begin for the solution/coding. Before you do, slow down a bit and do a little design.

- Have you seen similar problem before?
- Are there patterns that are recognizable in a potential solution?
- Is there existing software that implements the data, functions and features that are required?
- Has a similar problem been solved? If so, are the elements of solution reusable?
- Can subproblem be defined? If so, solutions are readily apparent for the subproblem?
- Can you represent a solution in a manner that leads to effective implementation? Can a design model be created?

3. Carry out the plan: The design you have created serves as a roadmap for the system you want to build. There may be unexpected detours and it is possible that you will discover an even better route as you go, but the ‘plan’ would allow you to proceed without getting lost.

- Does the solution conform to the plan?
- Is the source code traceable to the design model?
- Is each component part of solution provably correct?
- Has the design and code been reviewed?

- Have the correctness proofs been applied to the algorithm?

4. Examine the result: You cannot be sure that your solution is perfect but you can be sure that you have designed a sufficient number of tasks to uncover as many error as possible.

- Is it possible to test each component part of the solution?
- Has a reasonable test strategy been implemented?
- Does the solution produced results that conform to the data, functions and features that are required?
- Has the software been validated against all stakeholders' requirements?

05/08/2026

GENERIC PRACTICES FOR SOFTWARE ENGINEERING

David Hooker in 1996 has proposed seven principles that focus on software engineering practice as a whole. They are reproduced below:

1. The First Principle: The Reason It All Exists

- A software system exists for one reason: to provide value to its users.
- All decisions should be made with this in mind.
- Before specifying a system requirement, before noting a piece of system functionality, before determining the hardware platforms or development processes, ask yourself questions such as: "Does this add real value to the system?" If the answer is "no," don't do it.

2. The Second Principle: KISS (Keep It Simple, Stupid!)

There are many factors to consider in any design effort.

- All design should be as simple as possible, but no simpler.
- This facilitates having a more easily understood and easily maintained system. This is not to say that features, should be discarded in the name of simplicity.
- The more elegant designs are usually the more simple ones. Simple also does not mean "quick and dirty." It often takes a lot of thought and work over multiple iterations to simplify the design.
- The payoff is software that is more maintainable and less error-prone.

3. The Third Principle: Maintain the Vision

- A clear vision is essential to the success of a software project. Without conceptual integrity, a system threatens to become a patchwork of incompatible designs, held together by the wrong kind of screws.
- Compromising the architectural vision of a software system weakens and will eventually break even the well-designed systems. Having an empowered architect who can hold the vision and enforce compliance helps ensure a very successful software project.

4. The Fourth Principle: What You Produce, Others Will Consume

- Always specify, design, and implement knowing that someone else will have to understand what you are doing. The audience for any product of software development is potentially large. Specify with an eye to the users.
- Design, keeping the implementers in mind.
- Code with concern for those that must maintain and extend the system. Someone may have to debug the code you write, and that makes them a user of your code. Making their job easier adds value to the system

5. The Fifth Principle: Be Open to the Future

- In today's computing environments, where specifications change on a moment's notice and hardware platforms are obsolete just a few months old, software lifetimes are typically measured in months instead of years. However, true "industrial-strength" software systems must endure far longer times.
- To do this successfully, these systems must be ready to adapt to these and other changes. Systems that do this successfully are those that have been designed this way from the start.
- "Never design yourself into a corner". Always ask "what if," and prepare for all possible scenarios/answers by creating systems that solve the general problem, not just the specific ones.

06/08/2026

6. The Sixth Principle: Plan Ahead for Reuse

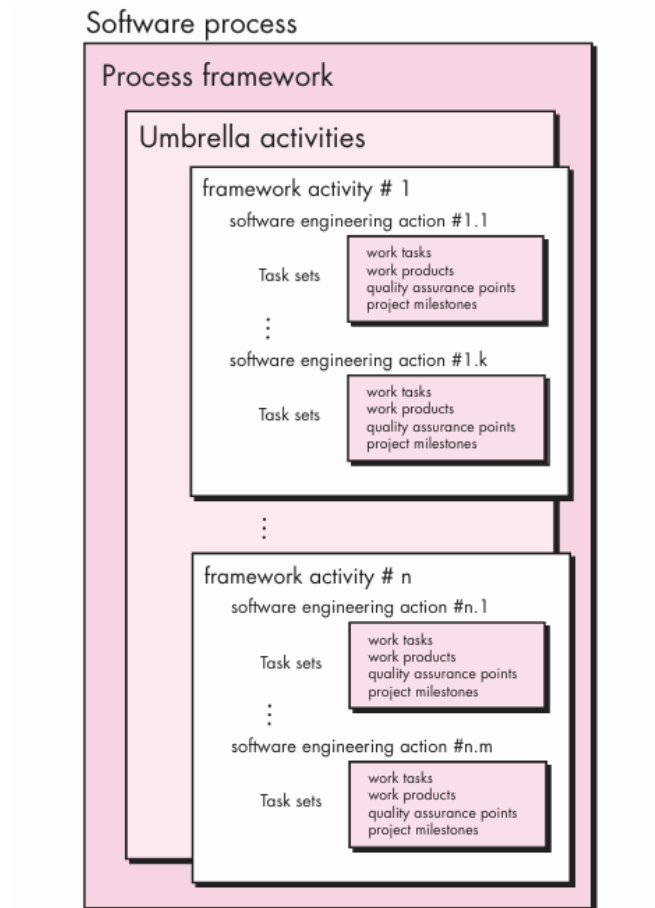
- Reuse saves time and effort. Achieving a high level of reuse is arguably the hardest goal to accomplish in developing a software system. The reuse of code and designs has been proclaimed as a major benefit of using object-oriented technologies.
- However, the return on this investment is not automatic. Planning ahead for reuse reduces the cost and increases the value of both the reusable components and the systems into which they are incorporated.

7. The Seventh principle: Think!

- This last principle is probably the most overlooked. Placing clear, complete thought before action almost always produces better results.
- When you think about something, you are more likely to do it right.
- You also gain knowledge about how to do it right again. If you do think about something and still do it wrong, it becomes a valuable experience. A side effect of thinking is learning to recognize when you don't know something, at which point you can research the answer.
- When clear thought has gone into a system, value comes out. Applying the first six principles requires intense thought, for which the potential rewards are enormous.

PROCESS MODELS

A process is a collection of activities, actions, and tasks that are performed to create work product. Each of these activities, actions, and tasks reside within a framework or model that defines their relationship with the process and with one another.



Each framework activity is populated by a set of software engineering actions. Each action is defined by a task set that identifies the work tasks that are to be completed, the work products that is to be produced, the quality assurance points that will be required, and the project milestones that will be used to indicate progress of the project.

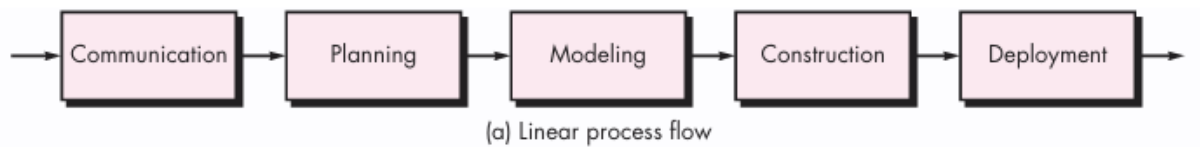
One of the important aspects of software process is process flow. It describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time.

07/08/2026

PROCESS FLOW

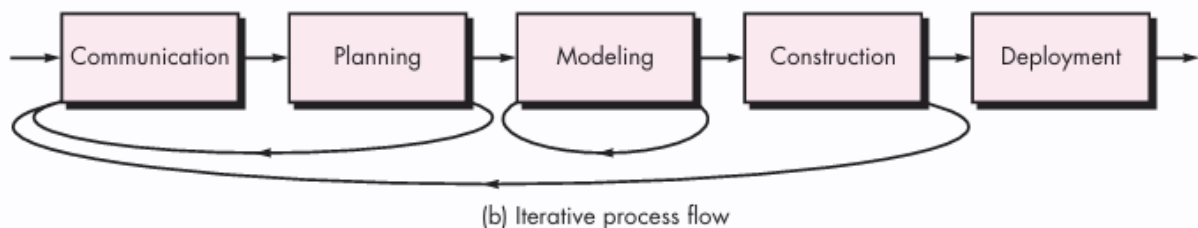
There is different organisation of process flow-

1. Linear Process Flow:



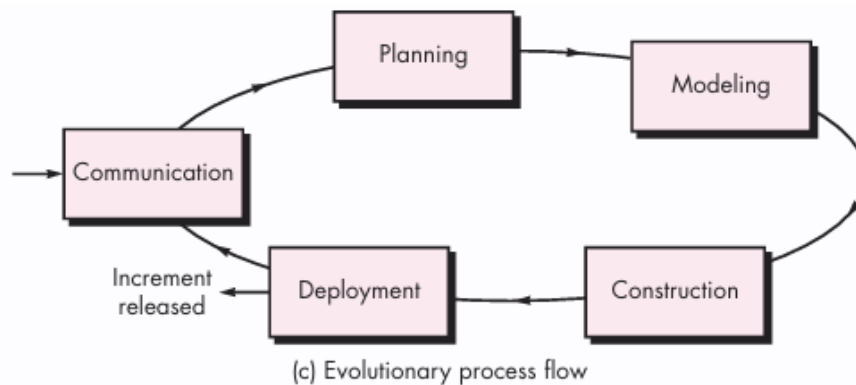
It executes each of the five framework activities in sequence, beginning from communication, then planning, then modelling, then construction and deployment. It is a software development approach where the development activities are performed one after another in a fixed sequence. Linear, as process generally moves forward from one phase to the next, rather than repeatedly going back and forth.

2. Iterative Flow:



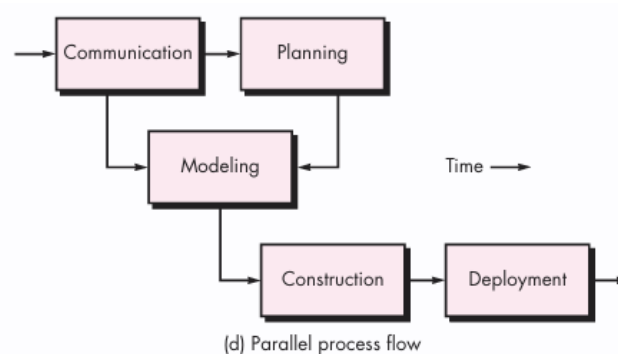
It repeats one or more activities before proceeding to the next. This concept is used in working interface.

3. Evolutionary Flow:



It executes the activities in a circular manner. Each circuit through the 5 activities leads to a more complete (better) version of software.

4. Parallel Process Flow:



It executes one or more activities in parallel with other activities.

For example: Modelling for one aspect of software might be executed in parallel with construction of another aspects of software.

PROCESS MODEL

They are fundamental, they represent the phases of a software development project.

The traditional models also called as prescriptive models. It defines a pre-defined set of process, elements and predictable process flow. We call them as “prescriptive” because they prescribe a set of process elements- framework activities, actions, tasks, work products, quality assurance and change control mechanism for each project.

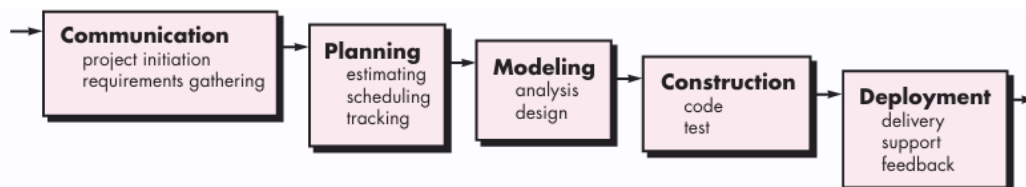
Each process model also prescribes a process flow (also called a work flow)—that is, the manner in which the process elements are interrelated to one another.

There are various models developed over the period of time.

THE WATERFALL MODEL

The waterfall model, sometimes called linear sequential model (the classic life cycle), suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through planning, modelling, construction, and deployment, culminating in ongoing support of the completed software. The waterfall model is the oldest paradigm for software engineering. It however has some issues.

1. Real projects rarely follow the sequential flow that the model proposes.
2. It is often difficult for the customer to state all requirements explicitly at the beginning of many projects.
3. The customer must have patience. A working version of the program(s) will not be available until late in the project time span.
4. A major bugs/blunder may not be detected until the working program is reviewed.
5. It works in a fast pace.
6. Takes minimum time among all models. However, whether software product is working properly or not.
7. It is used for comparing time taken by other models to make project.



12/08/2026

SOFTWARE DEVELOPMENT MODELS

Software Development Life Cycle (SDLC)

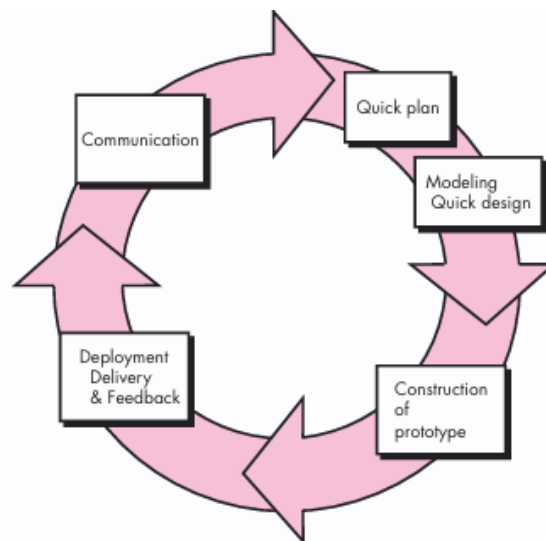
SDLC has generally seven phases: -

1. **Planning:** It identifies project scope, goals and requirements. It provides initial project plan.
2. **Analysis:** It gathers and reviews data and project requirements. It provides fully detailed requirement documentation.
3. **Design:** It defines the project architecture. It develops/provides software design document.
4. **Coding:** It writes the initial code and provides functional software prototype.
5. **Testing:** It reviews code and eliminates bugs. It provides a refined and optimized software.
6. **Deployment:** It deploys the code to production environment. It provides the software for the end users.
7. **Maintenance:** It continuously fixes and improves the software. It improves and optimizes the code.

PROTOTYPE PROCESS MODEL

A customer often defines a set of general objectives for software, but does not identify detailed requirements for functions and features. In some cases, the developer may be unsure of the efficiency of an algorithm, the adaptability of an operating system, or the form that human-machine interaction.

In such situations, a prototyping paradigm may offer the best options/approach. It helps developer and other stakeholders to better understand what is to be built when requirements are fuzzy. It is commonly used with other models.

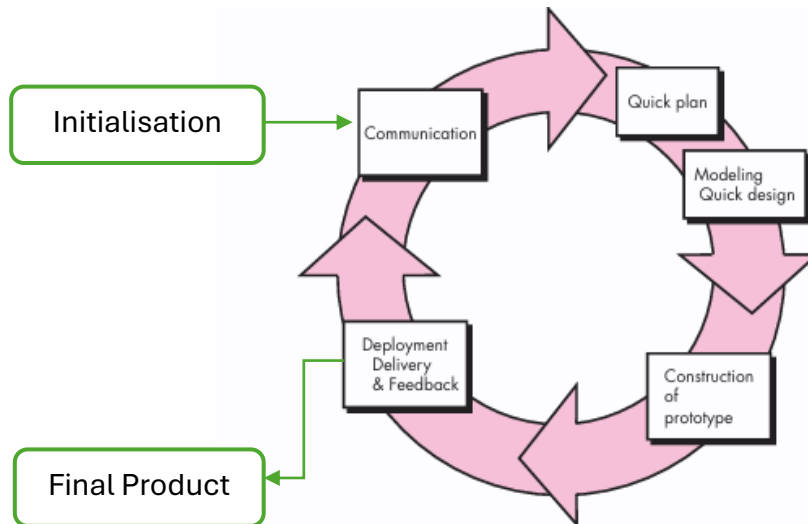


The prototyping paradigm begins with communication. We need to meet with the stakeholders to define the overall objectives for the software, identify whatever requirements are known.

A prototyping iteration is planned quickly, and modelling in the form of a “quick design” is done. A quick design focuses on a representation of those aspects of the software that will be visible to end users (e.g., human interface layout or output display formats). The quick design leads to the construction of a prototype. The prototype is deployed and evaluated by stakeholders, who provide feedback that is used to further refine requirements.

ITERATION MODEL

In this model, the software is dropped and refined through repeated cycle, called iteration. Each iteration passes through communication, planning, modelling, construction and deployment phase until the software is complete or ready for release. In this model, errors are found and fixed in early cycles rather than at the very end. It is easier to adapt change of user needs or new technology. It however requires heavy resources.

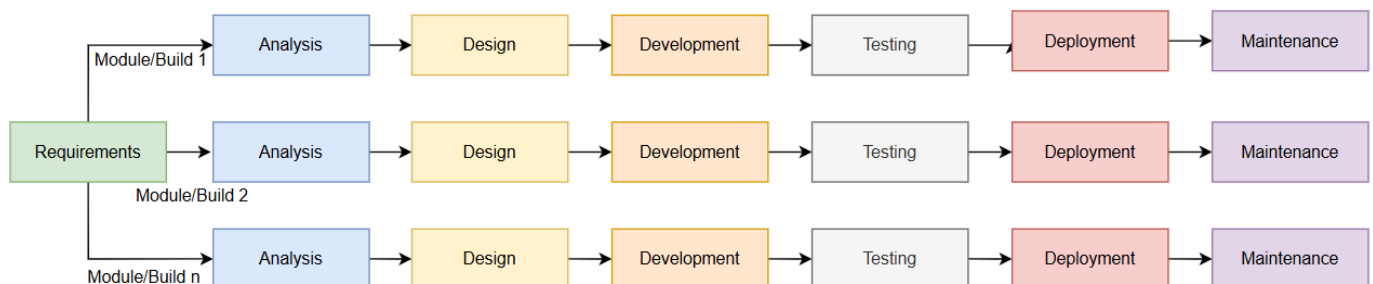


13/08/2026

INCREMENTAL MODEL

In this model, we first take some requirements and based on them create a module or function and delivers to the customer. Then, we take some new requirements and based on them create a new module and combine with the earlier ones.

This process is continued till the final project is delivered to the customer.

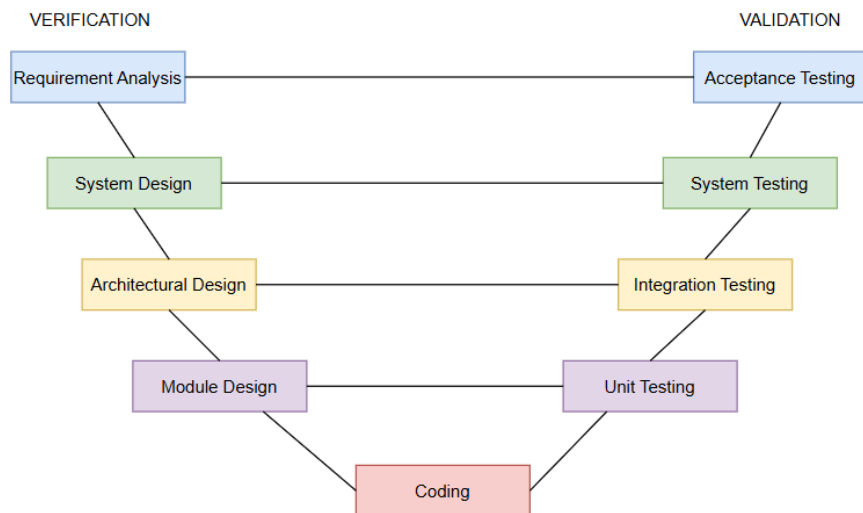


It however requires clear understanding for making the parts to development the complete system.

V-MODEL

It is also called verification and validation model.

The execution of each phase/process is sequential that is, a new phase/process is started only when the previous process is complete.



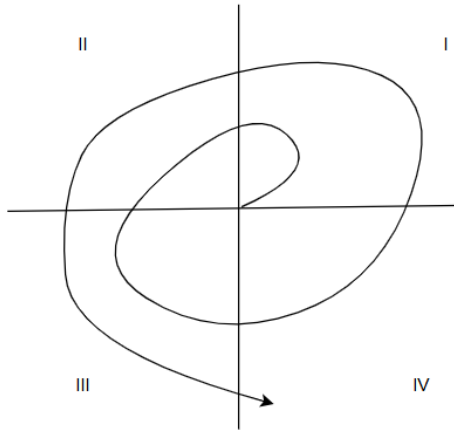
It is based on the association of testing phase with each development phase that is in V model shape. Both software development and testing take place simultaneously through coding lying at bottom.

SPIRAL MODEL (Risk based approach)

The entire process of this model is described in four phases which are repeated until the project is complete.

1. Identifying and resolving the risk
2. Determine the objectives and alternative solutions
3. Development and testing
4. Review and plan for next phase

In this model, the risks are identified in each phase and they are resolved. It uses the approaches from other models. So, it is also called Meta model. It was developed by Barry Boehm in 1996.



RAPID APPLICATION DEVELOPMENT (RAD) MODEL

It was developed to overcome the rigidity of waterfall model that makes difficult to incorporate the changes. It has the features of both prototyping and evolutionary models.

It deploys the evolutionary approach to obtain and incorporate the customer's feedback. The prototypes are constructed and incrementally the features are developed and delivered to the customers.

In this model the prototypes are not discarded but they are enriched and used in software developments.

Its major goals are

1. To decrease time and efforts/cost incurred to develop the software.
2. To limit the cost of accommodating the change requests.
3. To reduce the communication gap between the customers and developers.

RAD emphasizes code reuse as an important for completing a project faster.

It advocates the use of specialised tools to facilitate fast development of working prototype.

It is especially helpful for specific use cases where user interface or requirements need to drive development.

Being able to experience an interface, even a work in progress allows users to provide more useful feedback earlier in the development phase.